

Using the CRA to Understand Systems in Practice

ORCWG · Open Regulatory Compliance Working Group

Paula Mesa Macías

Founder, Pau&Company

www.pau.company



Pau&Company

Ethical technology consultancy

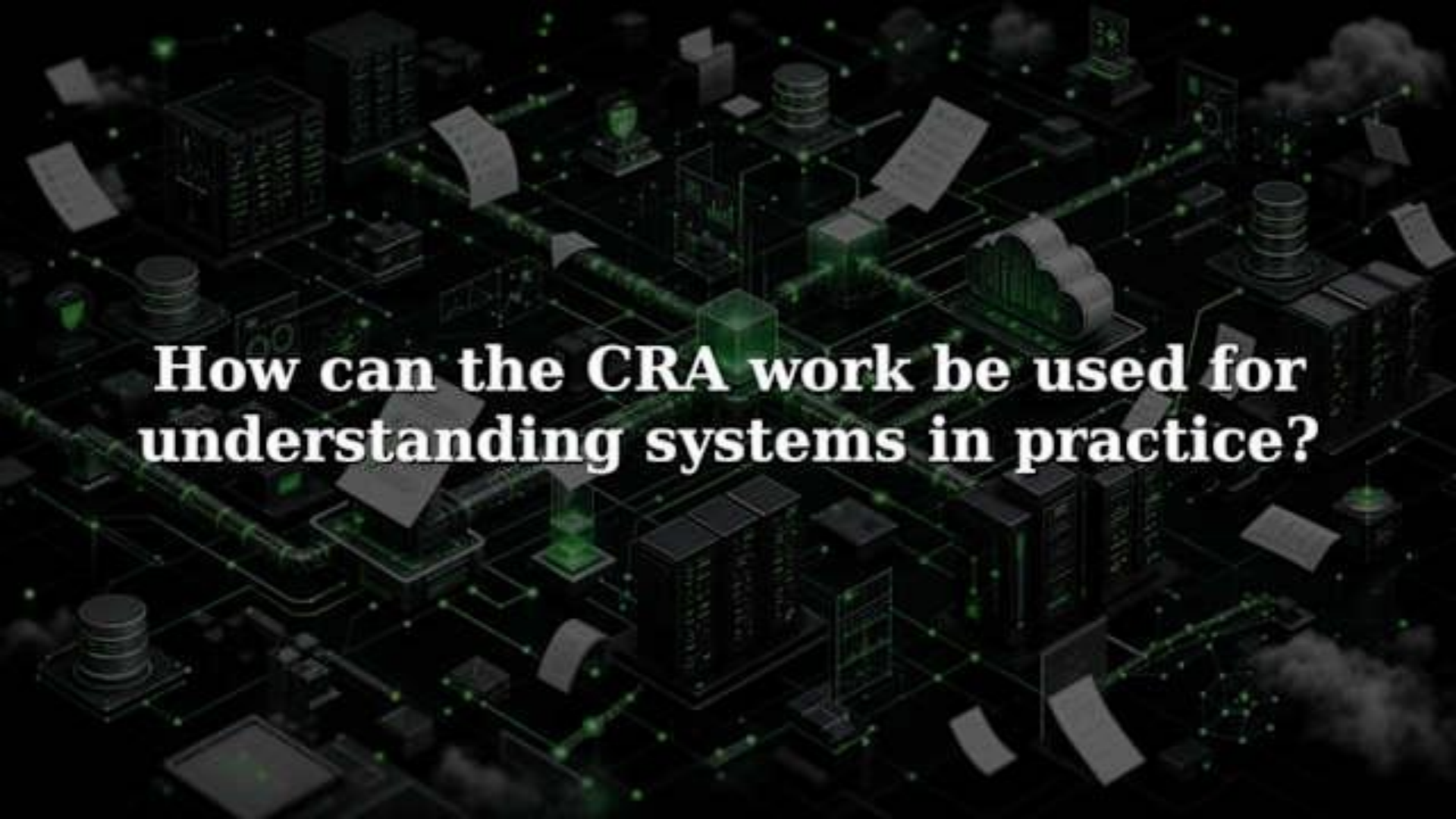
What we do

- 1 Analyse technical systems
- 2 Identify risks
- 3 Implement technical solutions
- 4 Provide training

Four focus areas

- Cybersecurity**
Security analysis, vulnerability management, CRA readiness
- Privacy**
Data protection, GDPR compliance, privacy-by-design
- Environmental impact of IT**
Carbon footprint of infrastructure, sustainable technology decisions
- Use of AI**
Risk assessment, governance, and responsible deployment of AI systems





How can the CRA work be used for understanding systems in practice?

In this session

01 Fragmented information

02 CRA and operational visibility

03 CRA and decision-making

04 Ethical Software Framework

Part one

Fragmented information

Why knowledge about a system disperses over time — and why it matters.

Knowledge disperses as systems grow

How complexity accumulates

deadline
immediate
problem
delivery
target
operational
issue

Different teams hold different pieces

Development

- ✓ How the application works day to day.
- ✗ How hard it is to maintain securely over time.

Infrastructure

- ✓ Deployment environments in detail.
- ✗ The risk that dependency decisions create.

Security

- ✓ Where vulnerabilities are.
- ✗ Who owns the fix, and what else could be affected.

So what happens to all that information?

Technically, the information exists.
But in practice, it is fragmented.

Having information somewhere is not the same thing as understanding how the system actually works.

In open source, this is even more pronounced

- 1 Knowledge is distributed across organisations, time zones, and years.

Contributors come and go. There is rarely a single person who sees the whole picture.

- 2 A component may be maintained by one person.

That knowledge lives in their head. If they leave, it leaves with them.

- 3 The ecosystem depends on people it has never met.

Maintaining things it cannot fully see.

Documentation is not the core issue

Documentation does matter

- ✓ Writing things down consistently does help.
- ✓ Documentation discipline reduces knowledge loss.
- ✓ It makes onboarding faster.
- ✓ It makes audits possible.

But documentation lags behind reality

- Every dependency update changes the system.
- Every configuration change changes the system.
- Every pipeline change changes the system.
- Documentation represents what someone understood at the moment they wrote it down.

The system changes continuously. Documentation, by its nature, does not keep up.

Informal knowledge is real — and fragile

What it is

Conversations

Decisions made in calls, in Slack, in passing.

Institutional memory

What the long-timers know about why things are the way they are.

People

The ones who remember why that decision was made three years ago.

Why it is fragile

→ It leaves when people leave.

→ It does not travel across organisational boundaries.

→ It cannot be shared with a new maintainer, a security auditor, or a regulator.

This creates real operational problems

When there is no coherent picture of the system:

Worse decisions

Dependencies that matter get missed. Risks go unnoticed.

Underestimated change

The spread of a modification is not seen until it is too late.

Invisible debt

Technical debt accumulates silently until it becomes a crisis.

Slower incidents

Response slows down because time is spent working out what is connected to what.

Part two

CRA and operational visibility

What the CRA requires — and why it's interesting

SBOM

Software Bill of Materials

A structured, machine-readable inventory of all components — versions, licences, known vulnerabilities. Not just a compliance document: a map of your system as it exists right now.

Vulnerability Management

Ongoing monitoring + defined response

Detect new CVEs, assess severity in context, deliver patches reliably. Reveals the gap between the security posture you claim and what you can actually demonstrate.

Lifecycle Documentation

Security properties + design assumptions

Document the threat model, intended deployment environment, maintenance lifecycle. Forces articulation of things that otherwise live only as informal knowledge.

Incident Notification

Vulnerabilities exploited in the wild

Building this process surfaces monitoring gaps and unclear escalation paths — operational readiness issues that may have existed for years.

Taken together: not just compliance documentation — operational intelligence.

Open source already produces most of this data

What's often missing is not the data itself — it's the connection between the data.

Already produced as standard practice

Release notes

What changed and when.

Security advisories

Known vulnerabilities and their status.

Dependency manifests

What the project depends on.

Changelogs

A record of decisions over time.

What the S-BOM adds

Aggregation

Brings all these sources together across the whole product, including transitive dependencies.

Structure

Makes existing sources coherent in a new way. Asks you to see the whole picture, not just the individual pieces.

Two things worth separating

The C R A is sometimes talked about as a burden on open source. That concern is real and deserves to be taken seriously.

The compliance burden

- Liability questions for open source maintainers.
- Documentation requirements that small projects may struggle to meet.
- Real resource costs for teams with limited capacity.

These concerns are real.

The value of the information

- The information compliance work generates is genuinely useful — beyond the regulation.
- It builds a more coherent picture of the system.
- That picture has operational value that does not expire when the audit ends.

Investing in this information has returns beyond compliance.

Part three

CRA and decision-making

Connecting the pieces

A pattern seen fairly often in this kind of work

First SBOM

Aggregates dependency data across full product surface for the first time

Discovery

One library: single maintainer, no release in 18 months, 2 open CVEs (medium severity)

Context

Transitive dependency of a component touching the authentication flow

The point

None of this was secret. CVEs were in NIST's National Vulnerability Database. No one had connected the pieces.

*“The challenge is not producing more information.
The challenge is connecting the information you have.”*

What do you do with that information?

That's where the interesting work starts.

1 Is it actually exploitable here?

Assess whether the C V E is exploitable in your specific deployment context — not just in the abstract.

2 What if the library stops being maintained?

Is there a viable replacement? How much work is migration? Is there a fork with active maintenance?

3 What is the priority?

Address it now, monitor it, or accept it as a known risk — each is a legitimate outcome, but it has to be a decision.

4 Document the decision.

The C R A requires you to demonstrate that your vulnerability management process is real and not just theoretical.

Fragility is not evenly distributed

It clusters.

Certain components, certain integrations, certain people — an outsized amount of risk depends on a small number of things.

These concentrations are often not visible until you aggregate information from multiple sources.

When you put these together, the picture becomes much more realistic than any single source would give you.

What to aggregate

S-BOM

What components exist, what they depend on, and what vulnerabilities are known.

Incident history

Where things have broken before, and which components were involved.

Maintenance patterns

Who is updating what, how frequently, and with what level of testing.

Analysis over time

The value is not in any single snapshot. It is in the picture it builds over time.

The first time

Discovery work.

Finding things you did not know were there.

Valuable — but limited to what exists right now.

The second time — and beyond

Transformative.

→ You have a baseline. You can see what has changed.

→ You can see which risks were addressed and which have drifted.

→ You can see new dependencies that have accumulated.

→ You can see whether the decisions made last time were the right ones.

One-off, informal knowledge simply cannot provide this.

Trade-offs

Trade-offs

Dependency
selection

Short-term appeal

Fast integration · large user base ·
good docs

vs

Longer-term risk

Security history · single maintainer ·
licence risk

Cloud infra

Operational convenience

Mature tooling · easy scaling ·
smooth integrations

vs

Concentrated risk

Provider lock-in · pricing
dependency · carbon footprint

Automation

What you gain

Reduces error, improves
consistency, faster delivery

vs

What changes

Shallower system understanding ·
harder to diagnose failures

“These trade-offs exist whether or not you name them. The question is whether you made the choice consciously.”

What the C R A makes visible

Vulnerability management

The security implications of dependency choices become concrete, not abstract.

S-BOM

Supply chain dependencies are visible in a way they often were not before.

Lifecycle documentation

Assumptions about deployment environment and maintenance that previously lived only informally now have to be articulated.

What the C R A doesn't cover

It's focused on security and resilience. These dimensions are largely outside its scope.

1

Environmental impact

The C R A does not systematically address the environmental cost of software and infrastructure decisions.

2

Equity and access

Who can actually use the software, under what conditions, and with what limitations — this is outside scope.

3

Organisational concentration

Questions of dependency on a small number of organisations in the open source ecosystem are not addressed.

These are real dimensions of technical decision-making.

So the question becomes:

Is there a way to structure the analysis of these broader trade-offs alongside the security and resilience work that the C R A drives?

That's what I want to talk about next.

Part four

Ethical Software Framework

Why knowledge about a system disperses over time — and why it matters.

Ethical Software Framework

Technical decisions increasingly affect multiple dimensions at once.

What technical decisions now affect

- Security
- Privacy
- Environmental impact
- Resilience
- Maintainability
- Operational transparency
- Legal obligations

In practice

These areas are usually analysed separately. Engineers are left balancing all of it under time pressure.

Purpose of the framework

Not to replace engineering judgement or existing standards.

To provide a structured way to analyse technical decisions when these dimensions interact with each other.

The C R A generates operational system knowledge

One of the reasons the C R A is interesting in this context is that it forces organisations to generate operational system knowledge that can support this kind of analysis.

All of that information becomes useful beyond compliance itself.

Once you can actually see how systems behave operationally, you can start reasoning more clearly about trade-offs.

What it forces you to generate

Dependency structures

Lifecycle information

Maintenance processes

Supply chain visibility

Update responsibility

Incident handling

Trade-offs in practice

These are not abstract questions. These are technical decisions with broader consequences.

Improving security

may increase computational overhead

Increasing observability

may increase data retention

Reducing infrastructure duplication

may reduce environmental impact while also reducing resilience

Moving to a cloud managed service

may improve maintainability while increasing dependency concentration

Using a very large AI model

may improve capability while increasing infrastructure and energy costs

Not trying to produce perfect answers

In many cases there is no perfect answer.

The goal

To make trade-offs visible enough that organisations can make decisions consciously rather than accidentally.

Better system understanding is a prerequisite for better ethical analysis. When knowledge is fragmented, it becomes very hard to reason properly about impact.

Open source and responsible stewardship

Modern software systems are highly interconnected.

A decision made in one layer of the ecosystem can propagate across many organisations downstream. Understanding dependencies and maintenance relationships is not only a technical issue anymore.

It also becomes part of responsible system stewardship.

The framework is still evolving. If any of this connects with problems you are working on, or if there are dimensions you think the framework should cover that it doesn't yet address, I'd very much like to hear about it.

What coherent system knowledge enables

1 Better prioritisation

When you can see your full dependency surface, vulnerability exposure, and operational fragility in one place, you can direct limited engineering time where it will have the most impact.

2 Better communication

Structured, documented analysis gives you material to have productive conversations with leadership, funders, users, and regulators – without requiring them to take everything on faith.

3 Earlier detection of problems

Risks that are present but not visible become incidents. A coherent, maintained picture of your system makes them visible before that happens.

4 Institutional memory

A record not just of what the system is, but of why it is that way – valuable when the team changes, the threat landscape shifts, or a significant architectural investment needs to be justified.

pau.company

This is not a one-time exercise. It is an ongoing practice.

Starting out

No S-BOM in place, vulnerability management mostly informal, documentation scattered. We help build a structured starting point.

Parts already done

Compliance programme underway, some tooling in place. We help connect it to better technical decision-making and broader trade-off analysis.

If any of this resonates

Whether you are working through C R A readiness, building better operational visibility, or trying to be more conscious about the trade-offs in your technical decisions —

reach out.

These are hard problems. They benefit from being worked through with people who have seen a range of different contexts.

Three things

1

System knowledge is fragmented

Not because teams are negligent, but because systems are complex, distributed, and constantly changing. Knowledge naturally disperses across people, tools, and time. This is a real operational problem, not just a compliance problem.

2

C R A work generates operational visibility

Not as a side effect — as a direct consequence of doing the compliance work seriously. The ess-bom, vulnerability management, lifecycle documentation, incident notification. Each, done properly, produces structured information that makes your system more understandable.

3

Connecting this to decision-making

Through the kind of trade-off analysis the Ethical Software Framework is designed to support — creates the conditions for decisions that are more conscious, better communicated, and more resilient to the kinds of surprises that come from operating systems you don't fully understand.

The C R A is often talked about as a burden.

**The information that serious compliance work
generates is not just for the regulator.**

It's for you.

It's a more honest picture of the systems you're responsible for. And that's something worth having, regardless of what the regulation requires.